

Total Points
96.125 / 105 pts

Question 1

Q1 15 / 15 pts

✓ - 0 pts Correct

Question 2

Q2 10 / 10 pts

✓ - 0 pts Correct (talks about a stack overflow)

Question 3

Q3 10 / 10 pts

✓ - 0 pts Correct

Question 4

Q4 3 / 10 pts

✓ - 7 pts Vague answer with some correct understanding of dynamic libraries

Question 5

Q5 10 / 10 pts

✓ - 0 pts Correct

Question 6

Q6 15 / 15 pts

✓ - 0 pts Correct

Question 7

Q7 10 / 10 pts

✓ - 0 pts Correct

Question 8

Q8 10 / 10 pts

✓ - 0 pts Correct

Question 9

Q9

13.125 / 15 pts

✓ - 1.875 pts Part B: Incorrect Bit width or vague/incorrect meaning

Question 2 1, 6, and 9 are worth 15 points each. Other questions are 10 points each.

1. Convert the following C++ code to MIPS. Preserve the C++ behaviour.

underflow → lol

```
while( num <= 127 ) → same thing as NOT 127 < num
{
    num -= 20;
}
```

num is an unsigned char and its address is stored in \$s4.

lbu \$t0, 0(\$s4) # load num

addi \$t1, \$zero, 127 # store 127

addi \$t3, \$zero, -20 # store -20

LOOP:

sltu \$t2, \$t1, \$t0 # find if 127 < num

bne \$t2, \$zero, END # if it is, end loop

addi \$t0, \$t0, -20

j LOOP # loop again

END:

sbu \$t0, 0(\$s4) # store it back

2. Explain how nums from the code below can potentially crash the program. Explain the whole mechanism of the potential problem.

```
int main()
{
    double nums[100000000];
    /*rest of the code*/
}
```

nums is a very large array. It is a local variable so it will be stored in the stack.

However, since nums is very large and made up of doubles, it will occupy 800000000 continuous bytes.

The stack might overflow (run out of space) and this will prevent the program from running since it will be unable to store anything else in the stack.

3. What is data forwarding? Why do we need it? Describe briefly how it works.

Data forwarding is a solution to a specific type of hazard in pipelining. When using pipelining, multiple instructions may be executed at once, with all them being in different steps in the process. An issue might occur when two instructions are pipelined and one

the program from running since it will be unable to store anything else in the stack.

4. What is dynamic (a.k.a. shared) library? What are its advantages?

A dynamic library is a library stored on the hard disk, separate from any executable files. Advantages: it can be used by multiple programs and prevents the size of their executable files from being too big.

Disadvantages: it has to be loaded into RAM from the disk which is very slow.

You also have to make sure there are no version conflicts.

Finally, data forwarding comes in. It forwards the data from the first instruction to the second one, even if it's past the leading stage.

instruction might depend on the output of another. However, this instruction may already be past the leading stage when the earlier instruction finishes. This is where

5. There are three types of instructions on some CPU

	Type A	Type B	Type C
CPI (in cycle/instruction)	4	6	7

There are two code sequences

	Instruction counts by type (in instructions)		
	Type A	Type B	Type C
Sequence 1	6	2	2
Sequence 2	2	3	4

Which sequence executes faster? Support your answer with mathematical computations. Use measurement units. All measurement units must be simplified. Without a correct mathematical explanation the answer is worth 0 points.

Sequence 1

$$6 \text{ instructions} \cdot \frac{4 \text{ cycles}}{\text{instruction}} + 2 \text{ instructions} \cdot \frac{6 \text{ cycles}}{\text{instruction}} + 2 \text{ instructions} \cdot \frac{7 \text{ cycles}}{\text{instruction}} = 24 + 12 + 14 = 50 \text{ cycles}$$

Sequence 2

$$2 \text{ instructions} \cdot \frac{4 \text{ cycles}}{\text{instruction}} + 3 \text{ instructions} \cdot \frac{6 \text{ cycles}}{\text{instruction}} + 4 \text{ instructions} \cdot \frac{7 \text{ cycles}}{\text{instruction}} = 8 + 18 + 28 = 54 \text{ cycles}$$

Sequence 1 executes faster than sequence 2 because it takes less cycles to execute

6. Write a C++ function `int CountOnes(unsigned int num)` that takes a value `num` and returns a number of 1's bits in it.

```
int CountOnes(unsigned int num) {  
    int result = 0;  
    for (int i = 0; i < 31; ++i) {  
        if (num & 1 << i) {  
            ++result;  
        }  
    }  
    return result;  
}
```


7. Please see below a fragment of RAM. Mark in it all bytes that belong to BNE instruction. Then, write this instruction in form of a MIPS assembly language command. Addresses are not important here, so they are omitted. As always in our class, assume big-endian approach. Addresses grow top-down.

Values
10010110
10101001
11101110
00010110
01100110
11111111
11111001
00110011
01010101
11110001
11111111
10101000
10011000
00000000
01010011
00010000

match 8 →

this is the BNE instruction

5, 19, 6, some number (address)
↓ translate

bne \$s3, \$a2, LABEL-1

*shostak said to make up a label *

bne rs, rt, label

000101	5	rs	rt	Offset
	6	5	5	16

REGISTER NAME, NUMBER, USE, CALL CONVENTION

NAME	NUMBER	USE	PRESERVED ACROSS A CALL?
\$zero	0	The Constant Value 0	N.A.
\$at	1	Assembler Temporary	No
\$v0-\$v1	2-3	Values for Function Results and Expression Evaluation	No
\$a0-\$a3	4-7	Arguments	No
\$t0-\$t7	8-15	Temporaries	No
\$s0-\$s7	16-23	Saved Temporaries	Yes
\$t8-\$t9	24-25	Temporaries	No
\$k0-\$k1	26-27	Reserved for OS Kernel	No
\$gp	28	Global Pointer	Yes
\$sp	29	Stack Pointer	Yes
\$fp	30	Frame Pointer	Yes
\$ra	31	Return Address	Yes

8. I lost a stupid bet to a friend and now I must write my next C++ program without using `#include` in the whole project. How can I do it?

Copy the code from the files you are `#include`'ing and paste it directly where the `#include` is. That's what your precompiler does anyway so you should tell your friend to give a better challenge next time! ▽

9. Answer the questions below for the scenario when SW instruction is executed. Feel free to write on the back of this page if needed.

* keep note that I labelled which channel is green/red

A) What is the bit-width of the "red" channel? What's the meaning of data that goes through that channel?

32 bits. An offsetted address for instructions like SW, LW.

B) What is the bit width of the "green" channel? What's the meaning of data that goes through that channel?

32 bits. The result of an ALU operation. It is sent back to be written in a register.

C) Is the RegWrite input asserted (activated) or deasserted (deactivated)? Why? Correct explanation is required.

RegWrite is deasserted because SW instruction does not write to a register. It reads from a register and writes in memory.

D) Is "blue cross" or "violet triangle" input used for the multiplexor? Why? Correct explanation is required.

Blue cross because we directly send the immediate value offset to the multiplexer.



